

Tarea online

Título de la tarea: Jugadores de Futbol.

Unidad 5: Programación orientada a objetos en PHP.

Ciclo formativo y módulo: Ciclo DAW. Módulo DSW.

Curso académico: 2025-2026

Contenidos y resultados de aprendizaje.

Contenidos

1. Orientación a objetos en PHP.
 1. Características de orientación a objetos en PHP.
 2. Creación de clases.
 1. Creación de clases (I).
 2. Creación de clases (II).
 3. Creación de clases (III).
 3. Utilización de objetos.
 1. Utilización de objetos (I).
 2. Utilización de objetos (II).
 4. Mecanismos de mantenimiento del estado.
 5. Herencia.
 1. Herencia (I).
 2. Herencia (II).
 6. Interfaces.
 1. Interfaces (I).
 7. Ejemplo de POO en PHP.
 1. Ejemplo de POO en PHP (I).
2. Programación en capas.
 1. Namespaces en PHP.
 2. Gestionar Dependencias. Composer.
 3. Separación de la lógica de negocio.
 1. Separación de la lógica de negocio (I).
 4. Generación del interface de usuario.

Resultados de aprendizaje

- RA5: Desarrolla aplicaciones Web identificando y aplicando mecanismos para separar el código de presentación de la lógica de negocio.

1.- Descripción de la tarea

Caso práctico

Juan se va a encargar de desarrollar un proyecto para el equipo de futbol local. Para el proyecto va a tener que generar códigos de barras para identificar a cada jugador y hacer pruebas con datos ficticios para comprobar que todo funciona antes de entregar el proyecto.

Juan le propone a **Carlos** que le ayude.

-Carlos ¿Podríamos buscar en Packagist alguna aplicación los código de barras?

-Juan Efectivamente. Trabajaremos también con Blade, para ahorrar código y buscaremos alguna aplicación para generar los datos de prueba.

Jugadores.

Deseamos diseñar una aplicación para gestionar los jugadores de un equipo de futbol. Para dicha aplicación trabajaremos con la tabla **jugadores**, para ella nos crearemos una base de datos nueva y un usuario con permiso en ella. Podemos reutilizar el usuario **gestor**, de otros ejercicios.

Se deja el enlace a "**tablas.sql**" con lo necesario para crear la base de datos, dar a **gestor** permiso en ella y crear la tabla **jugadores**.

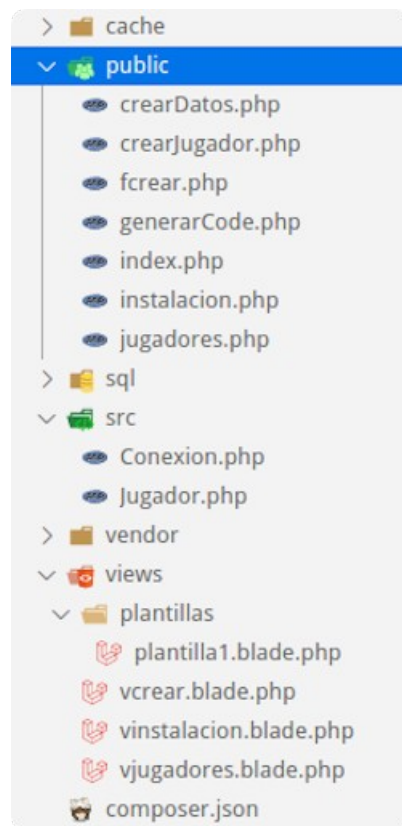
[Archivo SQL](#). (sql - 2 KB)

De los jugadores guardaremos nombre y apellidos, un teléfono, una nacionalidad (o país de nacimiento), una fecha de nacimiento, una posición, un número de dorsal (único) y un código de barras obligatorio único para cada uno.

Dependencias obligatorias a instalar con Composer.

- `philo/laravel-blade` como motor de plantillas.
- `milon/barcode` Para generar los códigos de barra (Se utilizara el formato EAN-13, que utiliza 13 números).
- `fzaninotto/faker` Para genera datos aleatorios de prueba.
- `autoload` con optimizador.

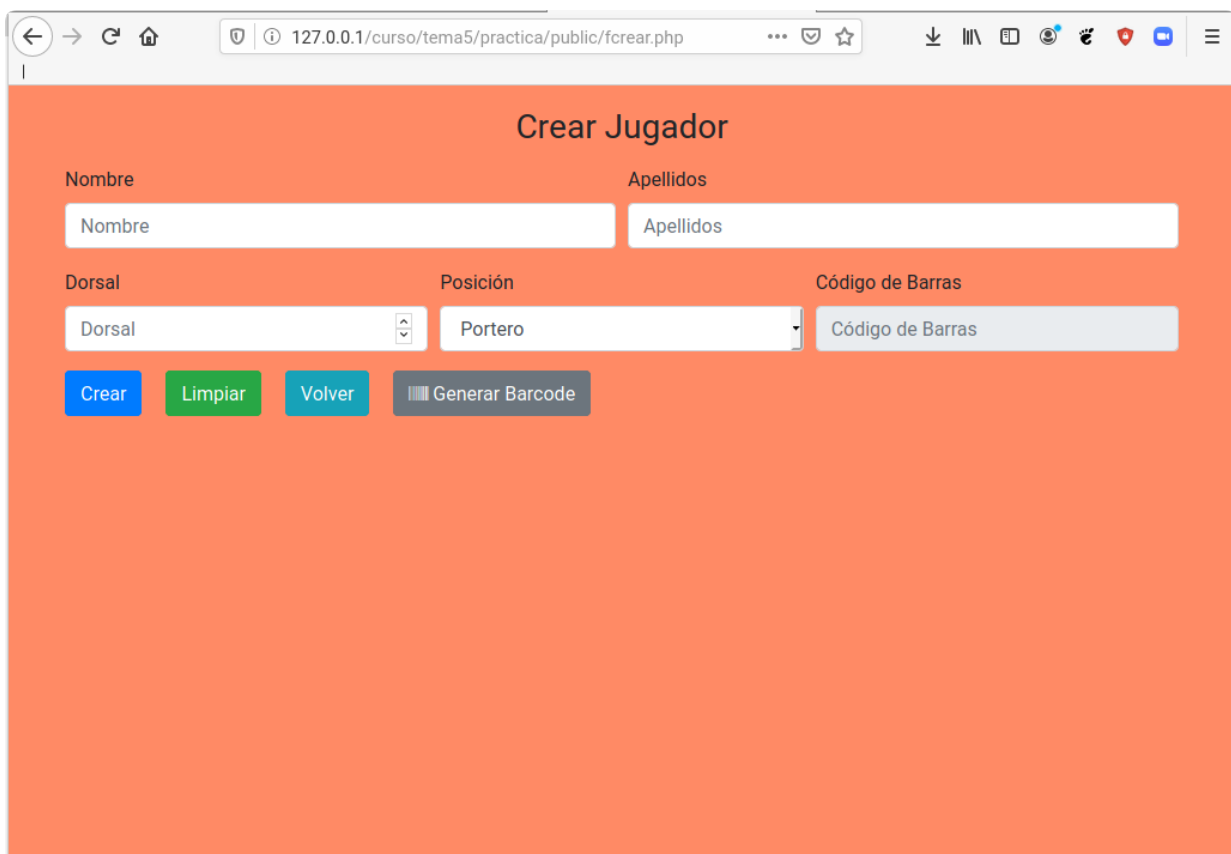
En la siguiente imagen podrás ver una estructura del proyecto ya terminada, lógicamente puedes usar los nombre de carpetas y ficheros que quieras:



1.- "cache" : Esta carpeta es necesaria crearla, y darle permiso "777" si estamos en Linux, para Blade y Barcode.

2.- "public": Aquí crearemos todas las páginas que visualizaremos desde el navegador o páginas ".php" para procesar algo. Las páginas que tendrá serán:

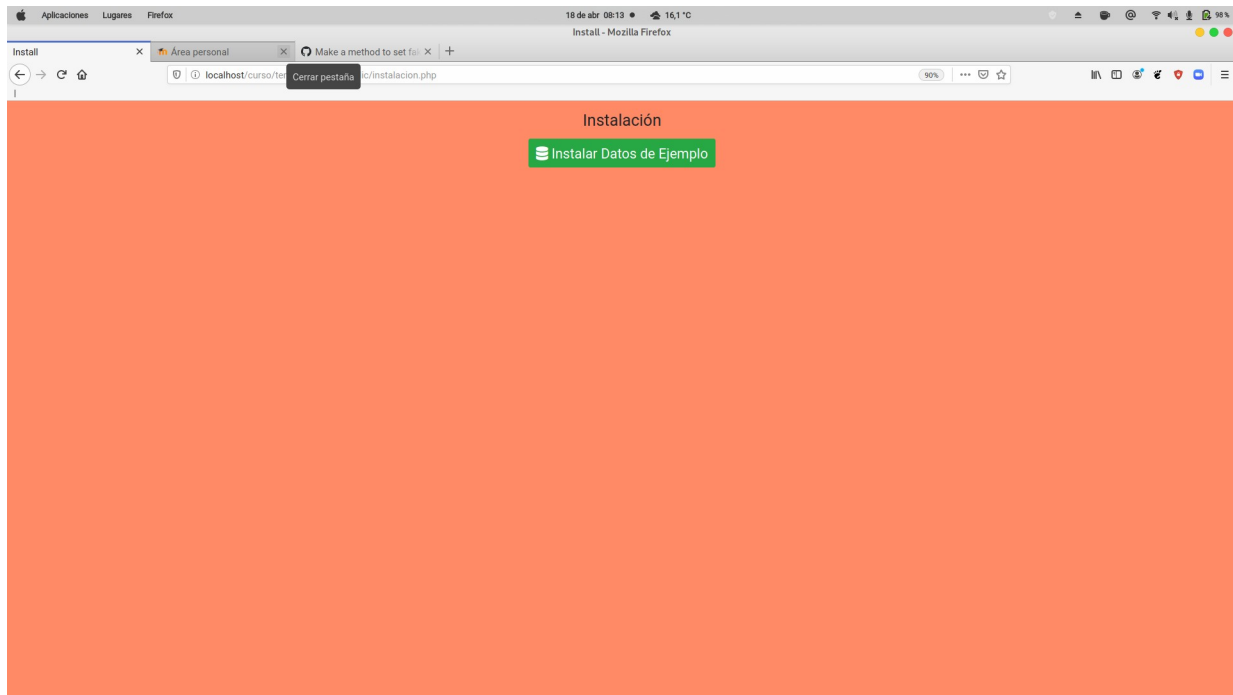
- "crearDatos.php" : Esta página me genera los datos de ejemplo.
- "crearJugador.php" : esta página es el "action" del formulario para crear un jugador. Controlaremos errores para no introducir un **dorsal** que ya existe, que **nombre**, **apellidos** y **nacionalidad** no esté vacíos.
- "fcrear.php": Es el formulario para crear el jugador. Llama a la vista: "vcrear.blade.php". A parte de los botones normales pondremos un botón para generar un código de barras válido ("href" a la página "generarCode.php"). El campo para el código de barra lo pondremos de sólo lectura (atributo "readonly")



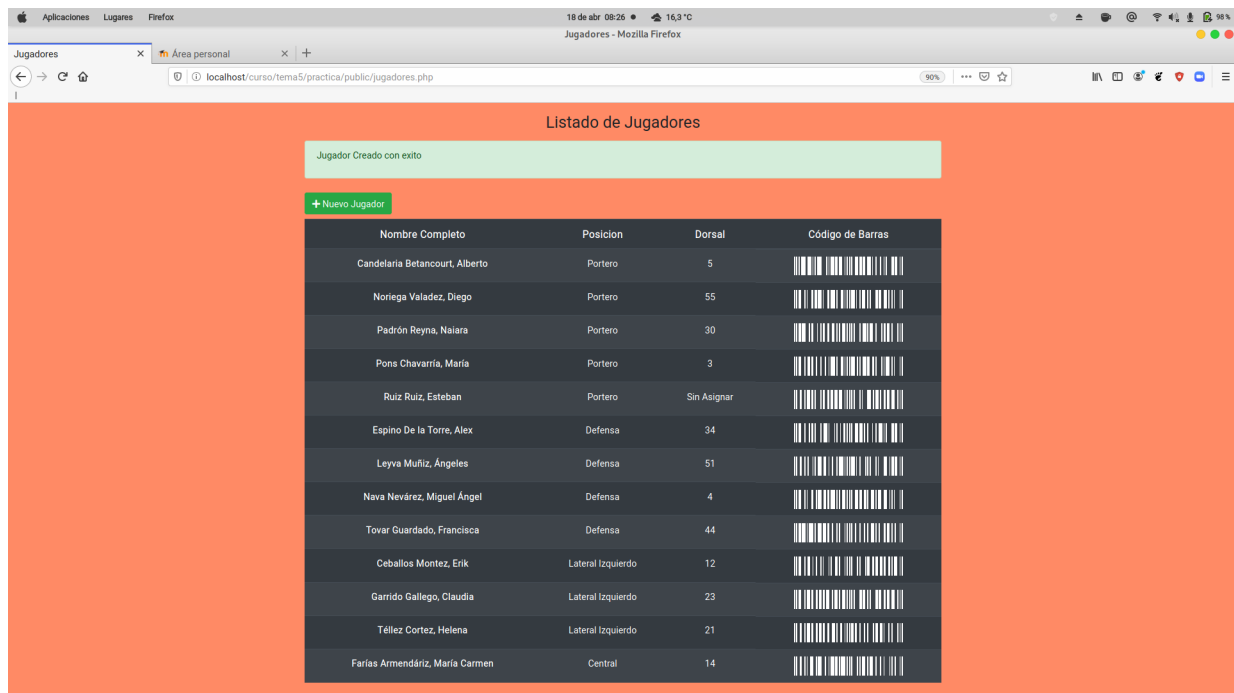
The screenshot shows a web browser window with the address bar displaying "127.0.0.1/curso/tema5/practica/public/fcrear.php". The page has an orange background and is titled "Crear Jugador". It contains a form with the following fields and buttons:

- Nombre**: A text input field with the placeholder "Nombre".
- Apellidos**: A text input field with the placeholder "Apellidos".
- Dorsal**: A text input field with the placeholder "Dorsal" and a small up/down arrow icon on the right.
- Posición**: A dropdown menu with "Portero" selected.
- Código de Barras**: A text input field with the placeholder "Código de Barras".
- Buttons**: Four buttons at the bottom: "Crear" (blue), "Limpiar" (green), "Volver" (teal), and "Generar Barcode" (grey with a barcode icon).

-
- "generarCode.php": Esta página me genera un código de barras EAN-13 válido y que además no exista en la base de datos.
- "index.php": Es la página de inicio, si la tabla jugadores no tiene datos llamará a "instalacion.php" para crearnos unos datos de ejemplo, si los tiene cargará la página "jugadores.php"
- "instalacion.php": Carga la vista "vinstalacion.php" básicamente un botón para ir a "crearDatos.php" y crearnos datos de ejemplo.



- "jugadores.php": Llama a la vista "vjugadores.php" Muestra en una tabla los datos de los jugadores. Tiene un botón crear que llama al formulario para crear un jugador nuevo. Si un jugador NO tiene **dorsal** mostraremos "Sin asignar".



3.- "sql" : Tiene el archivo ".sql", que se ha utilizado para crear la base de datos y la tabla. No es necesario para ejecutar la aplicación pero debe mantenerse en la entrega.

4.- "src": Contiene las Clases para gestionar la base de datos:

- "Conexion.php": Para crear la conexión.
- "Jugador.php": Para gestionar la tabla "**jugadores**". Aquí estarán los métodos para comprobar el "barcode", el **dorsal**, devolver los jugadores, insertarlos... Y todos los que sean necesarios.

5.- "vendor": La genera Composer.

6.- "views": Guardaremos todas las vistas ya mencionadas y la plantillas que utilizaremos.

A parte de todo esto está el archivo: "composer.json".

2.- Información de interés

Recursos necesarios y recomendaciones

Recursos Necesarios:

- Ordenador con un entorno AMP instalado y correctamente configurado, acceso a internet y un navegador.
- Xdebug instalado y configurado para PHP.
- Composer instalado
- Visual Studio Code instalado con las extensiones que hemos ido viendo, incluida la extensión para Xdebug.

Recomendaciones:

- Además del manual online de PHP, se recomienda dar libre acceso a Internet para la búsqueda de información.
- Infórmate de las versiones que instalas con Composer, instalar la última versión de las dependencias puede no ser una buena idea. Se deben instalar las versiones apropiadas para el correcto funcionamiento de la aplicación, comentar en el informe la seleccionada y el motivo. En la documentación de las librerías a ser instaladas puedes informarte de la mejor opción de la versión a ser utilizada.

Indicaciones de entrega

La entrega de la tarea es un archivo zip que contiene un informe que seguirá las pautas dadas y un directorio llamado **code** donde se pondrá toda la estructura generada para la aplicación.

Es importante indicar que en este informe se explicaran cada uno de los ficheros desarrollados, las diferentes versiones elegidas de las dependencias y su motivación.

Se establecen como pantallas obligatorias las siguientes: Estructura creada del proyecto con todas sus carpetas, archivos y subcarpetas, Formulario para crear el jugador, Muestra de datos creados

de **jugadores.php** y Una captura de **crearDatos.php** que verifique el uso correcto

de *fzaninotto/faker*

El envío se realizará a través de la plataforma de la forma establecida para ello, y el archivo se nombrará siguiendo las siguientes pautas:

3DAW_DSW_NombreApellido1_UT05_Tarea01.zip

3.- Evaluación de la tarea

Criterios de evaluación implicados

- Se han identificado las ventajas de separar la lógica de negocio de los aspectos de presentación de la aplicación.
- Se han analizado tecnologías y mecanismos que permiten realizar esta separación y sus características principales.
- Se han utilizado objetos y controles en el servidor para generar el aspecto visual de la aplicación Web en el cliente.
- Se han utilizado formularios generados de forma dinámica para responder a los eventos de la aplicación Web.
- Se han identificado y aplicado los parámetros relativos a la configuración de la aplicación Web.
- Se han escrito aplicaciones Web con mantenimiento de estado y separación de la lógica de negocio.
- Se han aplicado los principios de la programación orientada a objetos.
- Se ha probado y documentado el código.

Puntuación total 10 Puntos.

Rúbrica de la tarea	
Crea la estructura correcta del proyecto y se instalan correctamente las dependencias con Composer.	1.5 Puntos.
Se crean correctamente, con los métodos necesarios, las clases "Conexion" y "Jugador" y se hace uso de "namespaces" y herencia.	1.5 Puntos.
Crea correctamente los datos de Ejemplo con "fzaninotto/faker".	1 Punto.
Las páginas de la carpeta "public" cargan las vistas apropiadamente y les pasan los parámetros necesarios.	1 Punto.
El formulario para crear un jugador funciona correctamente, controlando posibles errores.	1 Punto.
Se muestran y generan correctamente los códigos de barra.	1 Punto.
Las vistas hacen uso de una plantilla común.	0.5 Puntos.
La página de inicio comprueba si existen datos o no en la tabla "jugadores" y hace las redirecciones adecuadas.	0.5 Puntos.
Se hace uso correcto de las sesiones para mostrar mensajes.	0.25 Puntos.
El diseño es adecuado y el código está comentado.	0.25 Puntos.
Mejoras aportadas, debidamente explicadas y justificadas.	0.5 Puntos.
Respetar las normas de presentación de las tareas.	1 Puntos.

